CS/ECE 752
Fall 2025
Homework 8
Due 11 AM Central Time on Friday, November 14th, 2025

NAME: _____

You should do this assignment on your own, although you are encouraged to talk with classmates electronically or on Piazza about any issues you may have encountered. The standard late assignment policy applies: you may submit up to 1 day (24 hours) late without penalty. Moreover, please consult the homework page for details about LLM usage on homework assignments.

If at all possible, please retain the format of the provided PDF/Word document. I am going to grade this homework using Gradescope, which works best when answers are in set places.

What to Hand In

To submit your assignment, please type up your answers to the following questions and submit **one PDF named HW8-<netID>.pdf** on Canvas. If you prefer writing your answers by hand, that is fine, but please scan your solutions and submit them on Canvas. **Please make sure to put your name in the top-right of the first page of the PDF. And please have the answer to each question start on a new page to simplify grading (like prior homeworks).** Note that just because each question starts on a new page, it does not mean you need to write a whole page of answer – the page breaks are there for simplifying grading.

Total Points: 55

Problem 1 [10 points]

Part A [5 points]

In class we discussed how perfect/infinite hardware does a much better job of exploiting ILP than "real" hardware. Give two reasons why this is the case. To receive credit, you much justify your answer.

Part B [5 points]

Why is hardware usually able to exploit more ILP in floating-point programs than integer programs? To receive credit, you must justify your answer.

Problem 2 [20 points]

Part A [12 points]

You were recently hired at a big computer architecture firm, MBI! As your first task at your new job, you are assigned to optimize code where the following loop is the most important piece of the program:

```
for (i = 0; i < 127; ++i)
     for (j = 0; j < 31; ++j)
          A[i][j] = B[i] * C[j];
```

To improve performance, you decide to add software prefetching support to prefetch the references you expect to miss in the cache. Assume that a read prefetch is expressed as *PREFETCH(&variable)* and it fetches the entire cache line in which the variable resides in shared mode. A read-exclusive prefetch operation, which is expressed as *PREFETCH_EX(&variable),* fetches the line in exclusive mode (i.e., so you can write it). Moreover, make the following assumptions about your system:

- Cache block size: 16 B.
- The cache is large enough to avoid any conflict or capacity misses.
- Each element of arrays A, B, and C is 4 B.
- Each iteration takes 16 cycles.
- Cache miss latency is 64 cycles; cache hit latency is 1 cycle.
- Size of array A is 128x32, B is 128, and C is 32.
- A[i][j] and A[i][j+1] are contiguous in memory, and so are A[i][31] and A[i+1][0].
- If necessary, you may assume A, B, and C are allocated such that they will never be on the same cache line (i.e., you may assume A[127][31] and B[0] are on different cache lines).
- You may assume the cost of adding prefetch instructions is minimal (i.e., each iteration still takes 16 cycles).

You may also consider using loop unrolling if you think it would be beneficial.

Rewrite the code to include as many prefetch operations as you believe are necessary to hide cache misses. Beneath the updated code, or inline with the code using comments, explain why you are adding each prefetch (i.e., what is each prefetch attempting to do).

(next page blank to provide space for this)

Part B [4 points]

What problem does prefetching attempt to tackle? Can prefetching hurt performance? To receive credit, you must justify your answer.

Part C [4 points]

Is prefetching more effective for single-issue, statically scheduled processors or multi-issue, dynamically scheduled processors? To receive credit, you must justify your answer.

Problem 3 [10 points]

In a virtually indexed, physically tagged cache, the cache set to search is selected using only bits of the virtual address, so virtual-to-physical address translation can proceed in parallel with reading tags for comparison.

In the simplest design, the associativity of the cache is large enough so that the cache index and offset bits together fit entirely into the page offset bits. However, page size remains relatively fixed with architectures while cache size grows with semiconductor technology so this may require a high associativity.

Part A [2 points]

A cache with a high associativity requires examining many ways on each access. This takes more time (even if comparisons are done in parallel) and uses more energy on each cache access, so we might want to keep associativity smaller. If a processor has 4KB pages and a 64KB level 1 cache, what is the minimum associativity required to use the simple virtually-indexed, physically tagged optimization?

Part B [2 Points]

Suppose a cache simply forms a longer index using a few of the least significant bits from the virtual page number. Describe a page table and access pattern where this cache will return incorrect data.

Part C [6 points]

Does the MIPS R10000 have problems with synonyms or homonyms (or both)? If so, how does it deal with them? If not, why not? To receive credit, you must justify your answer.

Hint: to answer this question, you may need to visit/revisit the MIPS R10K paper [Yeager '96] which was a "read" paper from earlier in the semester.

Problem 4 [15 points]

You are the lead DRAM architect at a company that builds accelerators. Your company is designing a new accelerator and needs you to determine what DRAM page policy (open vs. closed) the accelerator should use. The memory access pattern for the new accelerator is similar to this snippet:

```
int main(int argc, char * argv[])
{
      int arrSize = atoi(argv[1]);
      int offset = atoi(argv[2]);
      …
      int * arrA = (int *)malloc((arrSize + offset) * sizeof(int));
      int * arrB = (int *)malloc(arrSize * sizeof(int));
      int * arrC = (int *)malloc(arrSize * sizeof(int));
      …
      for (int i = 1; i < arrSize; ++i)
      {
            arrB[i] = arrC[arrA[i-1]] + arrA[(i + rand()) % offset];
      }
      …
}
```

Part A [10 points]

Assume that your system has no caches, and thus requests are sent directly from the processor to the memory controller. Moreover, assume that arrSize is large enough that the arrays cannot be in the row buffer at the same time, and that your memory controller can only handle 1 pending memory request at a time, so it cannot reorder memory requests. Finally, assume that for part A the main memory only has 1 bank and 1 DIMM. Which policy is likely to perform better? Again, to receive credit you must justify your answer.

Part B [5 points]

Now assume your memory has multiple banks/DIMMs. Which page policy will you select now, and why?